

C++ STUDY NOTES

CHAPTER 3

Inheritance; Polymorphism and Virtual Functions

Inheritance Types, Access Control, Virtual Base Classes, Abstract Classes,
Constructors/Destructors in Inheritance, Virtual Functions, Runtime Polymorphism

University : Rani Channamma University, Belagavi

Course Code : 24MSCCS 1.3

Programme : M.Sc. Computer Science

Semester : First Semester

Syllabus : 2024-25 onwards

Author : Sudarshan Badli

Academic Yr : 2026-2027

AUTHORSHIP

Prepared and Created by: Sudarshan Badli

© 2026 Sudarshan Badli. All Rights Reserved.

Table of Contents

3.1 Inheritance — Definition and Defining the Derived Class	3
3.2 Types of Derivation / Types of Inheritance	4
3.3 Public, Private and Protected Inheritance (Access Control)	7
3.4 Virtual Base Classes (solving the diamond problem)	8
3.5 Abstract Classes	9
3.6 Inheritance and Constructors	11
3.7 Inheritance and Destructors	12
3.8 Polymorphism — Pointers to Objects and Pointers to Derived Classes	13
3.9 Virtual Functions and Virtual Destructors	14
3.15 Examination Preparation — Chapter 3	18

HOW THIS CHAPTER IS ORGANISED

- Follows the official RCUB M.Sc. Computer Science syllabus (24MSCCS 1.3), Unit III, exactly — no topic is skipped or compressed.
- Each topic: Definition → Syntax → Key points → Diagram/Table (where relevant) → Full worked C++ program(s) with output.
- Sections 3.10–3.14 add deep-dive material: dry runs/trace tables, line-by-line explanation, advantages/disadvantages/applications, common mistakes, and head-to-head comparisons.
- Section 3.15 is a complete examination-preparation pack: 2/5/10-mark Q&A;,, programming questions, output prediction, find-the-error, MCQs and viva questions.

3.
1

Inheritance — Definition and Defining the Derived Class

DEFINITION

Inheritance lets a new class (the **derived** or child class) acquire the data members and member functions of an existing class (the **base** or parent class), then add its own members or redefine inherited ones. It models an "is-a" relationship (a Car is-a Vehicle) and is a key mechanism for code reuse.

SYNTAX

```
class Base {  
    // base class members  
};  
  
class Derived : access-specifier Base {  
    // derived class's own members, plus everything inherited  
};
```

KEY POINTS

- The **access-specifier** (public, protected, or private) between the colon and the base class name controls how the base class's public/protected members appear in the derived class (see Topic 3.3).
- The derived class constructor implicitly (or explicitly, via its initializer list) calls a base class constructor *before* running its own body.
- A derived class object contains a complete base-class sub-object plus its own additional members.

PROGRAM

```
#include <iostream>  
using namespace std;  
  
class Animal {  
public:  
    void eat() { cout << "This animal eats food.\n"; }  
};  
  
class Dog : public Animal {    // Dog inherits from Animal  
public:  
    void bark() { cout << "The dog barks.\n"; }  
};  
  
int main() {  
    Dog d;  
    d.eat();    // inherited from Animal  
    d.bark();   // Dog's own member  
    return 0;  
}
```

OUTPUT

```
This animal eats food.  
The dog barks.
```

3.
2

Types of Derivation / Types of Inheritance

DEFINITION

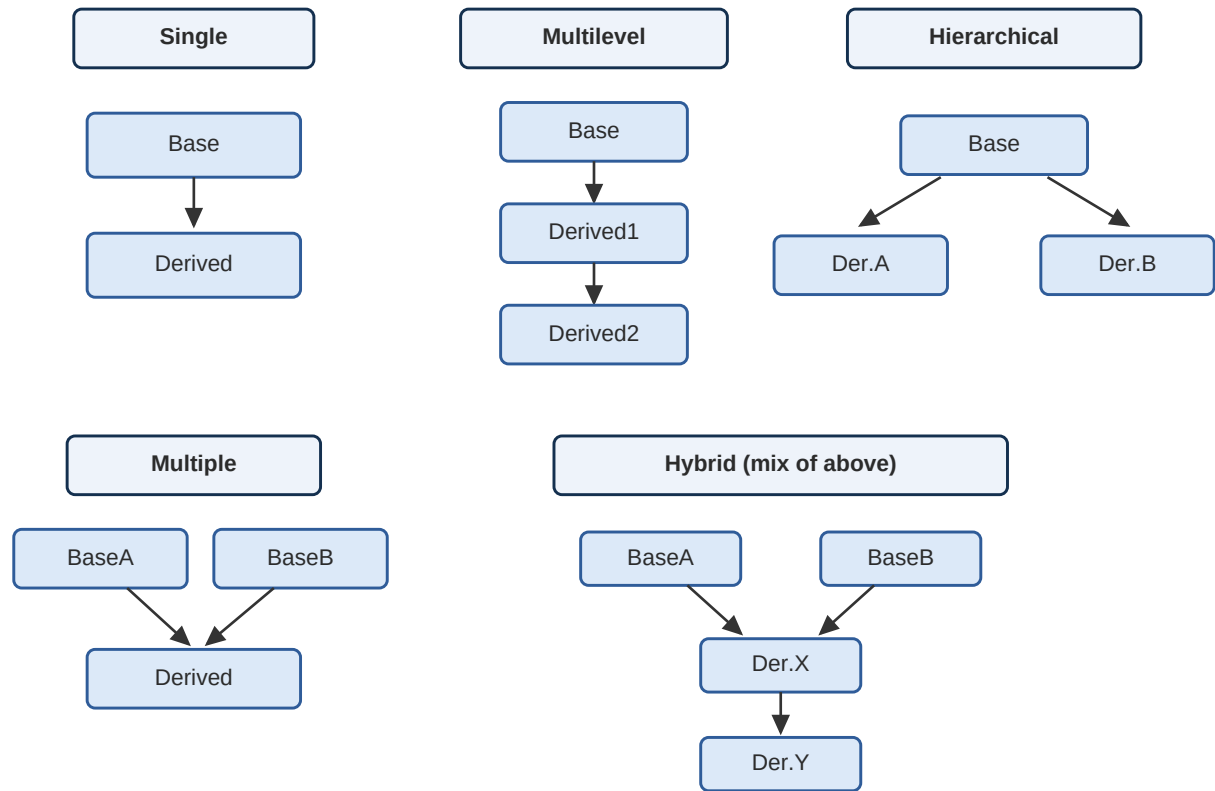
C++ supports several structural patterns of inheritance, depending on how many base and derived classes are involved and how they connect.

KEY POINTS

- **Single inheritance:** one derived class inherits from exactly one base class.
- **Multilevel inheritance:** a class is derived from a class that is itself derived from another (a chain: Base → Derived1 → Derived2).
- **Multiple inheritance:** one derived class inherits from *two or more* base classes directly.
- **Hierarchical inheritance:** multiple derived classes each inherit independently from the *same* single base class.
- **Hybrid inheritance:** a combination of two or more of the above patterns (e.g. hierarchical + multiple), which can introduce the diamond problem (see Topic 3.5).

FLOWCHART-STYLE CLASS DIAGRAM OF THE INHERITANCE PATTERNS

Types of Inheritance



PROGRAM – MULTILEVEL INHERITANCE

```
#include <iostream>
using namespace std;

class Animal {
public:
    void eat() { cout << "Eating...\n"; }
};
class Mammal : public Animal {
public:
    void walk() { cout << "Walking...\n"; }
};
class Dog : public Mammal {           // multilevel: Animal -> Mammal -> Dog
public:
    void bark() { cout << "Barking...\n"; }
};

int main() {
    Dog d;
    d.eat(); d.walk(); d.bark();      // all three levels accessible
    return 0;
}
```

OUTPUT

```
Eating...
Walking...
Barking...
```

PROGRAM – HIERARCHICAL INHERITANCE

```
#include <iostream>
using namespace std;

class Shape {
public:
    void info() { cout << "I am a shape.\n"; }
};
class Circle : public Shape {};      // Shape -> Circle
class Square : public Shape {};      // Shape -> Square (independently)

int main() {
    Circle c; Square s;
    c.info(); s.info();
    return 0;
}
```

OUTPUT

```
I am a shape.
I am a shape.
```

PROGRAM – MULTIPLE INHERITANCE

```
#include <iostream>
using namespace std;

class Teacher {
public:
    void teach() { cout << "Teaching...\n"; }
};
class Researcher {
public:
    void research() { cout << "Researching...\n"; }
};
class Professor : public Teacher, public Researcher {}; // two base classes

int main() {
    Professor p;
    p.teach(); p.research();
    return 0;
}
```

OUTPUT

```
Teaching...
Researching...
```

3.3

Public, Private and Protected Inheritance (Access Control)

DEFINITION

The access specifier written at the point of derivation controls how base-class members are inherited (NOT how members are declared inside the base class itself). It only ever makes access *more restrictive*, never less.

SYNTAX

```
class Derived : public Base { ... }; // most common
class Derived : protected Base { ... };
class Derived : private Base { ... };
```

Base member access	public derivation	protected derivation	private derivation
public	public	protected	private
protected	protected	protected	private
private	not inherited (inaccessible)	not inherited (inaccessible)	not inherited (inaccessible)

Example

```

#include <iostream>
using namespace std;

class Base {
public:
    int pub = 1;
protected:
    int prot = 2;
private:
    int priv = 3;          // never directly visible outside Base
};

class Derived : public Base {
public:
    void show() {
        cout << pub << " " << prot << "\n";    // OK: both accessible inside Derived
        // cout << priv;    would be an error - private members of Base are never
        //                    directly reachable in Derived, regardless of derivation type
    }
};

int main() {
    Derived d;
    d.show();
    cout << d.pub << "\n";    // OK: pub stays public with 'public' derivation
    return 0;
}

```

OUTPUT

```

1 2
1

```

3.4

Virtual Base Classes (solving the diamond problem)

DEFINITION

In **multiple inheritance**, if two base classes both derive from a common ancestor, and a class then inherits from both, the ancestor's members get duplicated — ambiguous. Declaring the shared base as a **virtual base class** ensures only one shared copy of it exists in the final derived object.

SYNTAX

```

class Base { ... };
class B1 : virtual public Base { ... };
class B2 : virtual public Base { ... };
class Derived : public B1, public B2 { ... };    // only ONE Base sub-object now

```

KEY POINTS

- Without virtual, Derived would contain two separate copies of Base, and an unqualified reference to a Base member would be ambiguous.
- This scenario is called the **"diamond problem"** because the inheritance diagram forms a diamond shape.

PROGRAM

```
#include <iostream>
using namespace std;

class Person {
public:
    string name = "Unnamed";
};
class Student : virtual public Person {};
class Employee : virtual public Person {};
class TeachingAssistant : public Student, public Employee {}; // diamond

int main() {
    TeachingAssistant ta;
    ta.name = "Asha";           // no ambiguity: only ONE Person sub-object
    cout << ta.name << "\n";
    return 0;
}
```

OUTPUT

Asha

3.
5

Abstract Classes

DEFINITION

An **abstract class** contains at least one **pure virtual function** (a virtual function with no body, marked = 0). It cannot be instantiated directly — its only purpose is to serve as a base/interface that derived classes must complete by overriding the pure virtual function(s).

SYNTAX

```
class Shape {                // abstract class
public:
    virtual double area() = 0; // pure virtual function
};

// Shape s;  error: cannot instantiate an abstract class
class Circle : public Shape {
public:
    double area() override { return 3.14159 * r * r; }
    double r;
};
```

KEY POINTS

- A class with even one pure virtual function is abstract; any class deriving from it is also abstract **unless** it overrides every pure virtual function.
- Abstract classes define a common **interface** that all derived classes must implement — central to runtime polymorphism.

PROGRAM

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual double area() = 0;    // pure virtual: makes Shape abstract
    virtual ~Shape() {}
};

class Circle : public Shape {
    double r;
public:
    Circle(double radius) : r(radius) {}
    double area() override { return 3.14159 * r * r; }
};

class Square : public Shape {
    double side;
public:
    Square(double s) : side(s) {}
    double area() override { return side * side; }
};

int main() {
    Shape *shapes[2];
    shapes[0] = new Circle(3);
    shapes[1] = new Square(4);
    for (int i = 0; i < 2; i++)
        cout << "Area = " << shapes[i]->area() << "\n";
    for (int i = 0; i < 2; i++) delete shapes[i];
    return 0;
}
```

OUTPUT

```
Area = 28.2743
Area = 16
```

3.6 Inheritance and Constructors

DEFINITION

When a derived object is created, the **base class constructor always runs first** (to build the inherited part of the object), then the derived class constructor runs (to build its own members). The derived constructor can pass arguments to a specific base constructor through its **member initializer list**.

SYNTAX

```
class Base {
public:
    Base(int x) { ... }
};
class Derived : public Base {
public:
    Derived(int x, int y) : Base(x) { // explicitly calls Base(x)
        // initialise Derived's own members using y
    }
};
```

KEY POINTS

- If the base class has no default constructor, the derived class **must** call a base constructor explicitly in its initializer list, or it will not compile.
- Construction order: Base class constructor(s) → member objects' constructors → Derived class constructor body.

PROGRAM

```
#include <iostream>
using namespace std;

class Base {
public:
    Base(int x) { cout << "Base constructor, x = " << x << "\n"; }
};
class Derived : public Base {
public:
    Derived(int x, int y) : Base(x) { // pass x up to Base
        cout << "Derived constructor, y = " << y << "\n";
    }
};

int main() {
    Derived d(10, 20);
    return 0;
}
```

OUTPUT

```
Base constructor, x = 10
Derived constructor, y = 20
```

3.7 Inheritance and Destructors

DEFINITION

Destructors run in the **reverse** order of construction: the derived class destructor runs first, then the base class destructor. If an object is deleted through a **base class pointer**, the base destructor must be declared `virtual`, otherwise only the base part is destroyed and the derived part leaks.

SYNTAX

```
class Base {
public:
    virtual ~Base() { ... }    // virtual: ensures correct cleanup via base pointer
};
class Derived : public Base {
public:
    ~Derived() { ... }
};
```

KEY POINTS

- Destruction order is always the reverse of construction order: most-derived class first, base class last.
- Rule of thumb: if a class has *any* virtual function, give it a virtual destructor too.

PROGRAM – WITHOUT VIRTUAL DESTRUCTOR — PROBLEM

```
#include <iostream>
using namespace std;

class Base {
public:
    ~Base() { cout << "Base destructor\n"; }    // NOT virtual
};
class Derived : public Base {
public:
    ~Derived() { cout << "Derived destructor\n"; }
};

int main() {
    Base *p = new Derived();
    delete p;    // only Base destructor runs: Derived destructor is skipped!
    return 0;
}
```

OUTPUT

```
Base destructor
```

PROGRAM – WITH VIRTUAL DESTRUCTOR — FIXED

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual ~Base() { cout << "Base destructor\n"; }    // virtual
};
class Derived : public Base {
public:
    ~Derived() { cout << "Derived destructor\n"; }
};

int main() {
    Base *p = new Derived();
    delete p;          // now both destructors run, in reverse order
    return 0;
}
```

OUTPUT

```
Derived destructor
Base destructor
```

3. Polymorphism — Pointers to Objects and Pointers to Derived Classes

DEFINITION

Polymorphism ("many forms") lets code written in terms of a base class work correctly with any derived class object. In C++, a **base class pointer (or reference) can point to a derived class object** — this is the foundation of runtime polymorphism, since the actual function called can be decided at run time based on the object's real type rather than the pointer's declared type.

SYNTAX

```
Base *bptr;
Derived d;
bptr = &d;          // a Base pointer validly pointing to a Derived object
bptr->function();    // which version runs depends on whether function() is virtual
```

KEY POINTS

- A base pointer can always point to a derived object (an "is-a" relationship upcast) — the reverse (derived pointer to base object) is unsafe and needs an explicit cast.
- Through a *non-virtual* function, the pointer's declared (static) type decides which version runs. Through a *virtual* function, the object's actual (dynamic) type decides — see Topic 3.9.

PROGRAM

```
#include <iostream>
using namespace std;

class Animal {
public:
    void speak() { cout << "Animal speaks\n"; }    // NOT virtual
};
class Dog : public Animal {
public:
    void speak() { cout << "Dog barks\n"; }
};

int main() {
    Animal *ptr;
    Dog d;
    ptr = &d;           // base pointer to a derived object
    ptr->speak();        // NOT virtual: calls Animal's version (static binding)
    return 0;
}
```

OUTPUT

```
Animal speaks
```

KEY POINTS

- Compare the output above with Topic 3.9's virtual-function example, where the output changes to "Dog barks" purely by adding the `virtual` keyword.

3.9

Virtual Functions and Virtual Destructors

DEFINITION

A **virtual function** is a base class member function declared with the `virtual` keyword, that a derived class can **override** (redefine with an identical signature). When called through a base class pointer/reference, the version that actually runs is chosen at **run time** based on the object's real type — this is **runtime (dynamic) polymorphism**, implemented internally using a vtable (virtual table).

SYNTAX

```
class Base {
public:
    virtual returnType functionName(parameters);
    virtual ~Base();           // virtual destructor, see Topic 3.7
};
class Derived : public Base {
public:
    returnType functionName(parameters) override; // override keyword is optional but recommended
};
```

KEY POINTS

- Once a function is declared `virtual` in the base class, it remains virtual in every further derived class automatically, even without repeating the keyword.
- The C++11 `override` keyword lets the compiler verify the signature truly matches a base virtual function, catching typos.
- A pure virtual function (`= 0`) is virtual with no default body, making the class abstract (Topic 3.5).
- Virtual function calls have a small runtime overhead (a vtable lookup) compared to ordinary calls.

PROGRAM – RUNTIME POLYMORPHISM VIA VIRTUAL FUNCTIONS

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void speak() { cout << "Animal speaks\n"; }    // virtual
    virtual ~Animal() {}
};

class Dog : public Animal {
public:
    void speak() override { cout << "Dog barks\n"; }
};

class Cat : public Animal {
public:
    void speak() override { cout << "Cat meows\n"; }
};

int main() {
    Animal *animals[3];
    animals[0] = new Animal();
    animals[1] = new Dog();
    animals[2] = new Cat();
    for (int i = 0; i < 3; i++)
        animals[i]->speak();    // correct version runs for EACH object
    for (int i = 0; i < 3; i++) delete animals[i];
    return 0;
}
```

OUTPUT

```
Animal speaks
Dog barks
Cat meows
```

Compile-time vs Runtime Polymorphism — Comparison

Point	Compile-time (static)	Runtime (dynamic)
Achieved by	Function overloading, operator overloading	Virtual functions + base class pointer/reference
Resolved	At compile time	At run time
Speed	Faster (no indirection)	Slightly slower (vtable lookup)
Also called	Early binding	Late binding

3.10 Deep Dive — Inheritance: Advantages, Disadvantages, Applications

Advantages, Disadvantages and Applications

ADVANTAGES

- Promotes code reuse: common functionality is written once in the base class and reused by every derived class.
- Supports a natural "is-a" modelling of real-world hierarchies (a Dog is-an Animal, a Car is-a Vehicle).
- Enables runtime polymorphism (Topic 3.9), letting new derived types be added without changing code that uses the base class interface.
- Reduces redundancy and makes large codebases easier to extend and maintain.

DISADVANTAGES

- Tight coupling: changes to a base class can unexpectedly break every derived class.
- Deep inheritance chains (multilevel) can become hard to understand and debug ("fragile base class" problem).

- Multiple inheritance can introduce ambiguity (the diamond problem), requiring virtual base classes to resolve.
- Overusing inheritance where simple composition ("has-a") would be clearer can lead to rigid, overly complex designs.

APPLICATIONS

- GUI frameworks: a generic Widget base class, with Button, TextBox, Checkbox as derived classes.
- Banking systems: a common Account base class with SavingsAccount and CurrentAccount derived classes sharing deposit()/withdraw() logic.
- Shape hierarchies in graphics/CAD software: an abstract Shape base class with Circle, Square, Triangle derived classes (Topic 3.5).
- Device driver hierarchies in operating systems, where a common interface is specialised per hardware type.

Line-by-Line Explanation

Line / Statement	Explanation
<code>class Animal {</code>	Begins the base class definition.
<code>public: void eat() { ... }</code>	A public member function available to Animal objects and to every class that inherits publicly from Animal.
<code>class Dog : public Animal {</code>	Begins the derived class Dog; <code>": public Animal"</code> means Dog inherits Animal's public members as public, and protected members as protected (Topic 3.3).
<code>public: void bark() { ... }</code>	Dog's own additional public member function, not present in Animal.
<code>Dog d;</code>	Creates a Dog object; internally it contains a complete Animal sub-object plus Dog's own members.
<code>d.eat();</code>	Calls the inherited eat() function — resolved by looking it up in Animal since Dog does not redefine it.
<code>d.bark();</code>	Calls Dog's own bark() function, which only Dog (and anything further derived from it) has.

COMMON MISTAKES / DEBUGGING TIPS

- Forgetting the access specifier in the derivation list (`class Derived : Base`) — for a `class`, this defaults to **private** inheritance, which is rarely what is intended; write `: public Base` explicitly.
- Assuming private members of the base class are inherited and usable in the derived class — they exist in the object's memory layout but are never directly accessible in the derived class, regardless of the derivation type (Topic 3.3).
- Using multiple inheritance from two classes that share a common ancestor without declaring that ancestor `virtual` — causes the diamond-problem ambiguity (Topic 3.4).
- Deleting a derived object through a base class pointer when the base destructor is **not** virtual — only the base part is destroyed, leaking the derived part's resources (Topic 3.7).

3.11 Deep Dive — Virtual Functions: Advantages, Disadvantages, Applications

Advantages, Disadvantages and Applications

ADVANTAGES

- Enable runtime polymorphism: one base-class interface, many correctly-chosen derived-class behaviours.
- Allow new derived classes to be added later without modifying the code that calls the virtual function through a base pointer/reference (the Open/Closed Principle).
- Make abstract base classes and clean interfaces possible via pure virtual functions (Topic 3.5).

DISADVANTAGES

- Small runtime overhead: a virtual call goes through a vtable (virtual table) lookup instead of a direct call.
- Every object of a class with virtual functions carries a hidden vtable pointer, slightly increasing object size.

- Constructors cannot be virtual, and calling a virtual function from inside a constructor/destructor does not behave polymorphically (it uses the current class's own version).

APPLICATIONS

- Plugin/driver architectures where new implementations are added without touching existing calling code.
- GUI event handlers (e.g. a virtual `onClick()` overridden differently by each button subtype).
- Shape/Animal-style exercises (Topic 3.9) that are extremely common in university exams.

DRY RUN — RUNTIME POLYMORPHISM PROGRAM (TOPIC 3.9)

Array index	Object actually created	animals[i]-> speak() resolves to	Output line
0	Animal()	Animal::speak() (vtable points here)	Animal speaks
1	Dog()	Dog::speak() (vtable points here)	Dog barks
2	Cat()	Cat::speak() (vtable points here)	Cat meows

Although the pointer's declared type is `Animal*` in all three cases, each object's own vtable entry — set at construction time — decides which `speak()` actually runs. This is exactly what "runtime" (dynamic) binding means.

3.12 Comparison — Inheritance Types at a Glance

Type	Base classes	Derived classes	Typical risk
Single	1	1	None special
Multilevel	1 (chain)	1 chain of several	Fragile base class if the chain is deep
Hierarchical	1	Several, independent	None special
Multiple	2 or more	1	Ambiguity if bases share an ancestor
Hybrid	Mixed	Mixed	Diamond problem — needs virtual base classes

3.13 Comparison — Virtual Function vs Pure Virtual Function

Point	Virtual function	Pure virtual function
Declaration	<code>virtual returnType fn();</code>	<code>virtual returnType fn() = 0;</code>
Function body	Has a default body in the base class	No body in the base class (= 0)
Can the class be instantiated?	Yes	No — makes the class abstract
Override required in derived class?	Optional (default behaviour used if not overridden)	Mandatory, unless the derived class is also abstract

3.14 Comparison — Overloading vs Overriding

Point	Overloading (Unit II)	Overriding (Unit III)
Classes involved	Same class / same scope	Base class and a derived class
Signature	Must differ (parameters)	Must be identical to the base version
Binding	Compile-time (static)	Run-time (dynamic), if the function is virtual
Keyword involved	None required	<code>virtual</code> (base), <code>override</code> (derived, optional but recommended)

3.
15

Examination Preparation — Chapter 3

NOTE ON QUESTION SOURCE

- The questions below are **expected / practice questions** prepared from the official RCUB syllabus for this chapter, in the common style of university exam papers.
- They are **not** claimed to be verified previous-year question papers — cross-check against your department's official previous-year papers for the exact verified questions, if available.

A. Two-Mark Questions and Answers

Q1. [2 marks] Define inheritance.

Inheritance is a mechanism by which a derived class acquires the data members and member functions of an existing base class, enabling code reuse and an "is-a" relationship.

Q2. [2 marks] What is multilevel inheritance?

A chain of inheritance where a class is derived from a class that is itself derived from another base class (Base -> Derived1 -> Derived2).

Q3. [2 marks] What is hierarchical inheritance?

A pattern where multiple derived classes each independently inherit from the same single base class.

Q4. [2 marks] What is the diamond problem?

An ambiguity that arises in multiple inheritance when two base classes share a common ancestor, causing the final derived class to contain duplicate copies of that ancestor's members.

Q5. [2 marks] How is the diamond problem solved?

By declaring the shared ancestor as a virtual base class in both intermediate base classes, ensuring only one shared copy exists in the final derived object.

Q6. [2 marks] What is an abstract class?

A class containing at least one pure virtual function (= 0); it cannot be instantiated directly and serves as an interface/base for derived classes.

Q7. [2 marks] What is a pure virtual function?

A virtual function declared with = 0 and no body in the base class, which every non-abstract derived class must override.

Q8. [2 marks] Define polymorphism.

The ability of the same function name, operator, or interface to behave differently depending on the object or context it is applied to.

Q9. [2 marks] What is the difference between compile-time and runtime polymorphism in one line?

Compile-time polymorphism (overloading) is resolved by the compiler before execution; runtime polymorphism (virtual functions) is resolved during program execution, based on the object's actual type.

Q10. [2 marks] Why should a base class destructor be virtual?

So that deleting a derived object through a base class pointer correctly calls the derived class destructor first, then the base destructor, avoiding resource leaks.

Q11. [2 marks] What access does "protected" inheritance give to inherited public members?

Public members of the base class become protected in the derived class, no longer directly accessible from outside the derived class.

Q12. [2 marks] Can a constructor be virtual? Can a destructor?

No, a constructor cannot be virtual. A destructor can (and, in most base classes, should) be declared virtual.

Q13. [2 marks] What is the order of constructor calls in inheritance?

The base class constructor always runs first, then the derived class constructor runs.

Q14. [2 marks] What is the order of destructor calls in inheritance?

The reverse of construction order: the derived class destructor runs first, then the base class destructor.

Q15. [2 marks] What is the override keyword used for?

An optional (but recommended) C++11 keyword placed on a derived class function to let the compiler verify it genuinely overrides a base class virtual function with a matching signature.

B. Five-Mark Questions and Answers**Q1. [5 marks]** Explain the types of inheritance in C++ with a diagram and one example each.

C++ supports five structural inheritance patterns. **Single**: one derived class, one base class (Dog : public Animal). **Multilevel**: a chain, Base -> Derived1 -> Derived2, e.g. Animal -> Mammal -> Dog, where Dog inherits members from both Mammal and Animal. **Hierarchical**: several derived classes independently inherit from one base, e.g. Circle and Square both deriving from Shape. **Multiple**: one derived class inherits from two or more base classes directly, e.g. Professor : public Teacher, public Researcher. **Hybrid**: a combination of the above, which can introduce the diamond problem if not handled with virtual base classes (Topic 3.4). The class-diagram figure in Topic 3.2 shows all five patterns side by side.

Q2. [5 marks] Explain public, protected and private inheritance with a comparison table and example.

The access specifier written at the point of derivation controls how base class members appear in the derived class, and can only make access more restrictive, never less. With **public** inheritance (most common), base public stays public and base protected stays protected in the derived class. With **protected** inheritance, both base public and base protected become protected in the derived class. With **private** inheritance, both become private in the derived class. In every case, the base class's own private members are never directly accessible in the derived class, regardless of the derivation type. The full table and worked program are in Topic 3.3.

Q3. [5 marks] What is a virtual base class? Explain the diamond problem and its solution with a program.

In multiple inheritance, if two base classes B1 and B2 both derive from a common class Base, and a further class inherits from both B1 and B2, the final object would contain two separate copies of Base's members, making any unqualified reference to them ambiguous — this is the "diamond problem", named after the shape of the inheritance diagram. Declaring Base as a **virtual base class** in both B1 and B2 (class B1 : virtual public Base) ensures the compiler keeps only one shared Base sub-object in the final derived class, resolving the ambiguity, as shown by the Person/Student/Employee/TeachingAssistant example in Topic 3.4.

Q4. [5 marks] Explain abstract classes and pure virtual functions with a program.

An abstract class is a class that contains at least one pure virtual function — a virtual function declared with "= 0" and no implementation in the base class. Because it is incomplete, an abstract class cannot be instantiated directly; it exists purely to define a common interface. Any class deriving from it is also abstract unless it overrides every pure virtual function it inherited. The Shape/Circle/Square example in Topic 3.5 demonstrates this: Shape declares virtual double area() = 0;, and only after Circle and Square each override area() with a real implementation can objects of those derived types be created and used polymorphically through Shape pointers.

Q5. [5 marks] Explain virtual functions and runtime polymorphism with a complete program and its output.

A virtual function is declared with the virtual keyword in the base class; when called through a base class pointer or reference, the version that actually executes is chosen at run time based on the object's real (dynamic) type, not the pointer's declared (static) type — implemented internally via a vtable. This is runtime (dynamic) polymorphism, in contrast to the compile-time polymorphism of function/operator overloading. The Topic 3.9 Animal/Dog/Cat example stores an Animal*, a Dog* and a Cat* (all created as new Dog(), new Cat() etc.) in an array of Animal* and calls speak() on each through the loop — producing "Animal speaks", "Dog barks", "Cat meows" respectively, even though every call site looks identical (animals[i]->speak()).

Q6. [5 marks] Discuss the construction and destruction order in inheritance, and explain why a base class destructor should be virtual, with a program.

When a derived object is created, the base class constructor runs first (building the inherited part), then the derived class constructor runs, adding its own members; the derived constructor can pass arguments up to a specific base constructor via its initializer list (Topic 3.6). Destruction happens in the exact reverse order: the derived destructor runs first, then the base destructor (Topic 3.7). If an object is deleted through a Base* pointer and the base destructor is NOT virtual, only the base part is destroyed and the derived part's destructor is skipped entirely, leaking any resources it owned — declaring the base destructor virtual fixes this, guaranteeing the full, correct destructor chain runs.

C. Ten-Mark / Long-Answer Questions**Q1. [10 marks]** Explain all types of inheritance supported in C++ in detail, with class diagrams, access rules and complete programs for at least three types.

Cover single, multilevel, hierarchical, multiple and hybrid inheritance as in Topic 3.2, reproducing the class-diagram figure, and explain that the access specifier used at derivation (public/protected/private, Topic 3.3) independently controls visibility regardless of which structural pattern is used. Provide complete, compilable programs for at least multilevel (Animal -> Mammal -> Dog),

hierarchical (Shape -> Circle, Shape -> Square) and multiple inheritance (Professor : public Teacher, public Researcher), each with its traced output, and discuss that hybrid inheritance (mixing patterns) can introduce the diamond problem, requiring virtual base classes (Topic 3.4) to resolve cleanly.

Q2. [10 marks] Discuss polymorphism in C++ in detail: compile-time vs runtime polymorphism, virtual functions, pure virtual functions, abstract classes and virtual destructors, with programs.

Polymorphism means the same interface behaves differently depending on context. Compile-time (static) polymorphism is achieved through function and operator overloading (Unit II), resolved entirely by the compiler before execution. Runtime (dynamic) polymorphism is achieved through virtual functions (Topic 3.9): declaring a base class function virtual lets a base class pointer/reference correctly call the derived class's overridden version, chosen at run time via a vtable. A pure virtual function (= 0) has no base implementation and makes its class abstract (Topic 3.5), unable to be instantiated, used purely to define a mandatory interface that every concrete derived class must implement — demonstrated by the Shape/Circle/Square example. Finally, explain why base class destructors should be virtual (Topic 3.7) whenever a class is meant to be used polymorphically through base pointers, with a before/after program showing the resource leak this prevents.

Q3. [10 marks] Explain access control in inheritance (public, protected, private derivation) with a complete comparison table and programs demonstrating all three.

Present the full derivation-access table from Topic 3.3: public derivation keeps base public members public and base protected members protected; protected derivation turns both into protected in the derived class; private derivation turns both into private; and in every case, base private members are never directly reachable from the derived class. Support this with three short programs, one per derivation type, each showing which members remain accessible from outside the derived class afterward, and explain the practical guideline that public inheritance ("is-a") is by far the most common choice, while private/protected inheritance ("implemented-in-terms-of") is rarer and used more deliberately.

Q4. [10 marks] Discuss the diamond problem in multiple inheritance in detail, explaining why it occurs, how virtual base classes solve it, and the construction order implications, with a complete worked program.

Explain that in ordinary (non-virtual) multiple inheritance, if two base classes B1 and B2 both derive from a common class Base, a class Derived : public B1, public B2 ends up containing two independent Base sub-objects, so code like derived.someBaseMember is ambiguous — the compiler cannot tell which copy is meant, giving a compile error. Declaring Base as virtual in both B1 and B2 (class B1 : virtual public Base) tells the compiler to allocate only one shared Base sub-object within any class that eventually inherits from both, resolving the ambiguity, as shown in the Person -> Student, Person -> Employee -> TeachingAssistant example (Topic 3.4). A thorough answer also notes the construction-order subtlety: with virtual inheritance, the *most-derived* class becomes responsible for initialising the shared virtual base, not the intermediate classes, which is a nuance often tested at this level.

Q5. [10 marks] Write and explain a complete program using an abstract Shape base class and at least three derived shape classes, demonstrating runtime polymorphism via a collection of base class pointers, with full output and a line-by-line explanation.

Design Shape with a pure virtual double area() = 0; and a virtual destructor; derive Circle, Square and Triangle, each overriding area() with its own formula and storing its own dimension(s). In main(), create an array (or a std::vector, foreshadowing Unit V) of Shape* pointing to dynamically created Circle, Square and Triangle objects, and loop through it calling shapes[i]->area() — explain that even though every call site is textually identical, the vtable set up for each concrete object ensures the mathematically correct formula executes for that object, which is the essence of runtime polymorphism. Conclude by deleting each pointer in a loop and explaining why the virtual destructor is essential here to avoid leaking the derived parts of each object.

D. Important Programming Questions

PRACTICE: WRITE THE FULL PROGRAM

- Write a C++ program implementing hierarchical inheritance with a Shape base class and Circle, Rectangle and Triangle derived classes, each with its own area() function (non-virtual).
- Write a C++ program demonstrating the diamond problem without virtual inheritance (showing the ambiguity error as a comment), then fix it using virtual base classes.
- Write a C++ program with an abstract Employee base class (pure virtual calculateSalary()) and derived classes Manager and Developer, demonstrating runtime polymorphism via a base class pointer array.
- Write a C++ program showing the difference in output between a virtual and a non-virtual destructor when a derived object is deleted through a base pointer.
- Write a C++ program demonstrating multilevel inheritance across four levels (e.g. LivingBeing -> Animal -> Mammal -> Dog), calling a member function from every level on one Dog object.

E. Output Prediction Questions

Q1. Predict the output:

```
#include <iostream>
using namespace std;

class Base {
public:
    void show() { cout << "Base"; }
};
class Derived : public Base {
public:
    void show() { cout << "Derived"; }
};

int main() {
    Base *b = new Derived();
    b->show();           // NOT virtual
    return 0;
}
```

ANSWER

Base

Explanation: Since `show()` is not virtual, the call is resolved using the pointer's declared (static) type, `Base*`, regardless of the object's actual type — so `Base::show()` runs, printing "Base".

Q2. Predict the output:

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual void show() { cout << "Base"; }
};
class Derived : public Base {
public:
    void show() override { cout << "Derived"; }
};

int main() {
    Base *b = new Derived();
    b->show();           // virtual
    delete b;
    return 0;
}
```

ANSWER

Derived

Explanation: Now `show()` is virtual, so the call is resolved using the object's actual (dynamic) type, `Derived`, via the vtable — `Derived::show()` runs, printing "Derived".

Q3. Predict the output:

```
#include <iostream>
using namespace std;

class A { public: A() { cout << "A "; } };
class B : public A { public: B() { cout << "B "; } };
class C : public B { public: C() { cout << "C "; } };

int main() {
    C obj;
    return 0;
}
```

ANSWER

A B C

Explanation: In multilevel inheritance, construction always proceeds from the topmost base class downward: A's constructor runs first, then B's, then C's own constructor body — giving "A B C" in that order.

F. Find-the-Error Questions

Q1. Identify and correct the error:

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual double area() = 0;
};

int main() {
    Shape s;           // ERROR
    return 0;
}
```

EXPLANATION

Shape is an abstract class (it has a pure virtual function with no override), so it cannot be instantiated directly. Only a fully-overriding derived class can be instantiated.

Q2. Identify and correct the error:

```
#include <iostream>
using namespace std;

class Base {
public:
    ~Base() { cout << "~Base"; }    // NOT virtual -- BUG, not a compile error
};
class Derived : public Base {
public:
    ~Derived() { cout << "~Derived"; }
};

int main() {
    Base *b = new Derived();
    delete b;           // only "~Base" prints -- Derived's destructor is skipped
    return 0;
}
```

EXPLANATION

This compiles and runs, but is a logic bug: because `~Base()` is not virtual, deleting through a `Base*` only calls `~Base()`, skipping `~Derived()` entirely and leaking any resources `Derived` owned. Fix: declare virtual `~Base()` {...}.

Q3. Identify and correct the error:

```
#include <iostream>
using namespace std;

class B1 { public: int x = 1; };
class B2 { public: int x = 2; };
class D : public B1, public B2 {};

int main() {
    D obj;
    cout << obj.x;    // ERROR: ambiguous
    return 0;
}
```

EXPLANATION

D inherits an x from both B1 and B2, so obj.x is ambiguous — the compiler cannot decide which one is meant. Fix: qualify explicitly, e.g. obj.B1::x or obj.B2::x.

G. Multiple Choice Questions (with explanations)

1. In which inheritance type does one class inherit from two or more base classes directly?

(A) Single (B) Multilevel (C) Multiple ✓ (D) Hierarchical

Answer: (C). Multiple inheritance is when one derived class inherits directly from two or more base classes.

2. What problem can arise from hybrid (combined) inheritance when two paths share a common ancestor?

(A) Stack overflow (B) The diamond problem ✓ (C) Infinite recursion (D) Memory leak

Answer: (B). When two base classes share a common ancestor and a class inherits from both, the ancestor's members get duplicated, causing ambiguity — the diamond problem.

3. How is the diamond problem resolved in C++?

(A) Using static members (B) Using friend functions (C) Using virtual base classes ✓ (D) Using operator overloading

Answer: (C). Declaring the shared ancestor as a virtual base class ensures only one copy of it exists in the final derived object.

4. A class containing at least one pure virtual function is called:

(A) A final class (B) An abstract class ✓ (C) A static class (D) A friend class

Answer: (B). Such a class is abstract: it cannot be instantiated and serves only as an interface/base.

5. What must a derived class do to become instantiable if its base class is abstract?

(A) Nothing special (B) Override every pure virtual function it inherited ✓ (C) Declare a static constructor (D) Make all members public

Answer: (B). A derived class remains abstract (and non-instantiable) until it overrides every pure virtual function from its base.

6. Which keyword, placed in the base class, enables runtime polymorphism for a member function?

(A) static (B) virtual ✓ (C) friend (D) const

Answer: (B). Declaring a function virtual in the base class allows derived classes to override it with run-time (not compile-time) resolution through a base pointer/reference.

7. What determines which overridden virtual function actually executes, when called through a base class pointer?

(A) The pointer's declared type (B) The object's actual (dynamic) type ✓ (C) The order of class declarations (D) The compiler's optimisation level

Answer: (B). With virtual functions, the object's real type (looked up via its vtable) decides which override runs, not the pointer's declared type.

8. Why should a base class intended for polymorphic deletion declare a virtual destructor?

(A) To save memory (B) To allow multiple destructors (C) So deleting via a base pointer correctly runs the derived class destructor too ✓ (D) It is required by the compiler for all classes

Answer: (C). Without a virtual destructor, deleting a derived object via a Base* pointer skips the derived class's destructor, leaking its resources.

9. Constructor call order in inheritance is:

(A) Derived first, then Base (B) Base first, then Derived ✓ (C) Both run simultaneously (D) Order is undefined

Answer: (B). The base class constructor always completes before the derived class constructor body runs.

10. Which of these can be overloaded, but NOT overridden?

(A) A virtual function in a derived class (B) A function in the same class with different parameters ✓ (C) A pure virtual function (D) A base class destructor

Answer: (B). Overloading happens within the same scope/class with differing parameter lists; overriding specifically involves a derived class redefining an identically-signed virtual base function.

H. Viva-Voce Questions**BE READY TO ANSWER VERBALLY, IN YOUR OWN WORDS**

- What is the "is-a" relationship, and how does inheritance model it?
- What access level do private base-class members have in a derived class?
- Why does class Derived : Base (no specifier) default to private inheritance?
- What is the practical difference between hierarchical and multiple inheritance?
- Why can a virtual base class not simply be declared normally to fix the diamond problem?
- Can an abstract class have a constructor? Can it have non-pure member functions?
- What happens if you try to create an object of an abstract class?
- What is a vtable, in your own words?
- Why can a constructor not be virtual?
- What is the difference between early binding and late binding?
- If a base class has a virtual function with a default implementation, must every derived class override it?
- What is the purpose of the override keyword, and is it mandatory?
- Why is it good practice to give a polymorphic base class a virtual destructor even if it currently holds no resources?
- Explain, with an example, why base.someMember can be ambiguous under multiple inheritance.
- What is the difference between function overriding and function hiding (redefining a non-virtual function)?

I. Chapter 3 — Revision Notes (Must-Remember Points)**TOP 10 EXAM-CRITICAL POINTS**

- Five inheritance patterns: single, multilevel, hierarchical, multiple, hybrid.
- Derivation access (public/protected/private) only ever makes access MORE restrictive, never less; base private members are never directly reachable regardless.
- class Derived : Base (no specifier) defaults to PRIVATE inheritance — always write "public" explicitly unless private/protected is intended.
- Diamond problem = duplicated common ancestor in hybrid/multiple inheritance; fixed with "virtual" on the shared base in both intermediate classes.
- Abstract class = has ≥ 1 pure virtual function ($= 0$); cannot be instantiated; derived class must override all of them to become concrete.
- Construction order: Base $\rightarrow$ Derived. Destruction order: Derived $\rightarrow$ Base (exact reverse).
- A base class meant for polymorphic use through pointers should have a VIRTUAL destructor.
- Virtual function + base pointer/reference = runtime polymorphism, resolved via vtable using the object's actual type.
- Non-virtual function + base pointer = static binding, resolved using the pointer's declared type (often a common exam trap).
- Overloading (same class, different parameters, compile-time) is NOT the same as overriding (base/derived, same signature, run-time via virtual).